

Детекция отмыывания денег на транзакционном графе

Домашнее задание №3, ML Start 2026

Тема: A.1, детектор финансового мошенничества (датасет согласован).

Датасет: IBM Transactions for Anti Money Laundering, файл `HI-Small_Trans.csv`.

Репозиторий: <https://github.com/USERNAME/REPO>

Бонусные пункты: SHAP и feature importance; blending и stacking; GitHub с README и requirements.txt; дополнительное исследование (error analysis по типам схем, ablation по группам признаков).

1 Постановка задачи

Задача: бинарная классификация банковских переводов на «отмывание / норма». Дана транзакция $x_t = (u \rightarrow v, a, t)$, требуется предсказать $y_t \in \{0, 1\}$.

Ключевая особенность данных это экстремальный дисбаланс: один позитив на 1122 транзакции. Отсюда выбор метрики. Ассигасу бесполезна, поскольку константный ответ «не отмывание» даёт 99.91%. ROC-AUC вводит в заблуждение, так как усредняет по всем порогам, включая заведомо нерабочие области, и остаётся высоким у практически непригодных моделей. Основная метрика в работе это **PR-AUC** (average precision), дополнительно везде указывается lift, то есть отношение PR-AUC к базовой частоте позитивов.

Второе ограничение это причинность. Признаки транзакции в момент t могут использовать только историю $\mathcal{H}_t = \{x_s : s < t\}$. Каждый построенный признак имеет вид

$$\phi(x_t) = g(\{x_s \in \mathcal{H}_t : \text{acct}(x_s) = \text{acct}(x_t), t - s < W\})$$

для окна $W \in \{1, 3, 7\}$ дней и агрегатора g .

2 Данные, EDA и предобработка

2.1 Описание датасета

Синтетические банковские логи, сгенерированные симулятором AMLSim. Исходно 11 колонок: временная метка, идентификаторы банков и счетов отправителя и получателя, суммы и валюты отправления и получения, формат платежа, метка класса.

Параметр	Значение
Транзакций (после усечения)	5 077 237
Позитивов	4 522 (0.089%)
Дисбаланс	1 : 1122
Уникальных счетов	496 995
Банков	30 470
Период наблюдения	10 суток (усечено с 18)
Признаков после feature engineering	86

Взята HI-версия (high illicit ratio), а не LI: при таком дисбалансе ещё более редкий позитивный класс сделал бы оценку неустойчивой. Файлы Medium и Large не помещаются в 12.7 ГБ RAM бесплатного Colab.

2.2 Предобработка и типы данных

Приведение типов даёт экономию памяти примерно в 15 раз против наивного `read_csv`:

- временная метка переведена в `int32`, минуты от начала наблюдения, что экономит около 150 МБ против `datetime64`;
- идентификаторы счетов закодированы **сквозной** нумерацией `int32`: отправители и получатели проходят через один `factorize` по объединённому столбцу. При отдельном кодировании один и тот же счёт получил бы разные ID в двух ролях, и графовые признаки потеряли бы смысл;
- валюты и формат платежа переведены в `category`, суммы в `float32`.

Пропусков в датасете нет. Выбросы обрабатываются винзоризацией по перцентилям 0.1% и 99.9%, причём квантили считаются только по обучающему периоду, чтобы не подглядывать в `holdout`. Это не косметика: суммы охватывают 18 порядков величины (от 10^{-6} до 10^{12}), и до винзоризации `lbfgs` не сходился ни на одном фолде, поскольку `StandardScaler` делил на раздутое выбросами стандартное отклонение.

Категориальные признаки обрабатываются двумя способами: `one-hot` для линейных и древесных моделей, обучаемые `embeddings` для MLP.

2.3 Три находки, определившие дизайн эксперимента

Краевой эффект симулятора. Легитимный трафик обрывается в середине 10-го дня, а транзакции отмывающих схем тянутся ещё неделю: в днях 10–17 остаётся 1108 транзакций, из них 655 позитивных, то есть доля отмывания 59% против 0.089% в основной части. Так выглядит край окна симуляции, где схемы, начатые под конец периода, доигрывают переводы после остановки фоновой генерации. Валидация на таком хвосте измеряла бы артефакт, поэтому данные усечены по день 9.

Недельная сезонность. Дни 2, 3 и 9 содержат около 207 тыс. транзакций против 482 тыс. в остальные, то есть это выходные. Доля позитивов там вдвое выше, но не потому, что в выходные больше отмывают: фоновый трафик падает, а генерация схем идёт равномерно. Отсюда два следствия: день недели включён в признаки, а фолды валидации нарезаны по целым дням.

Доминирование АСН. На этот канал приходится 11.8% транзакций и 86.6% всех позитивов. При этом у Wire и Reinvestment ровно ноль позитивов, как и у всех кросс-валютных переводов. Ноль подозрительно ровный: AMLSim просто не генерирует схемы через эти каналы. Формат платежа поэтому даёт почти детерминированный сигнал, которого в реальном банке не будет, и все метрики далее считаются в двух режимах, с ним и без него.

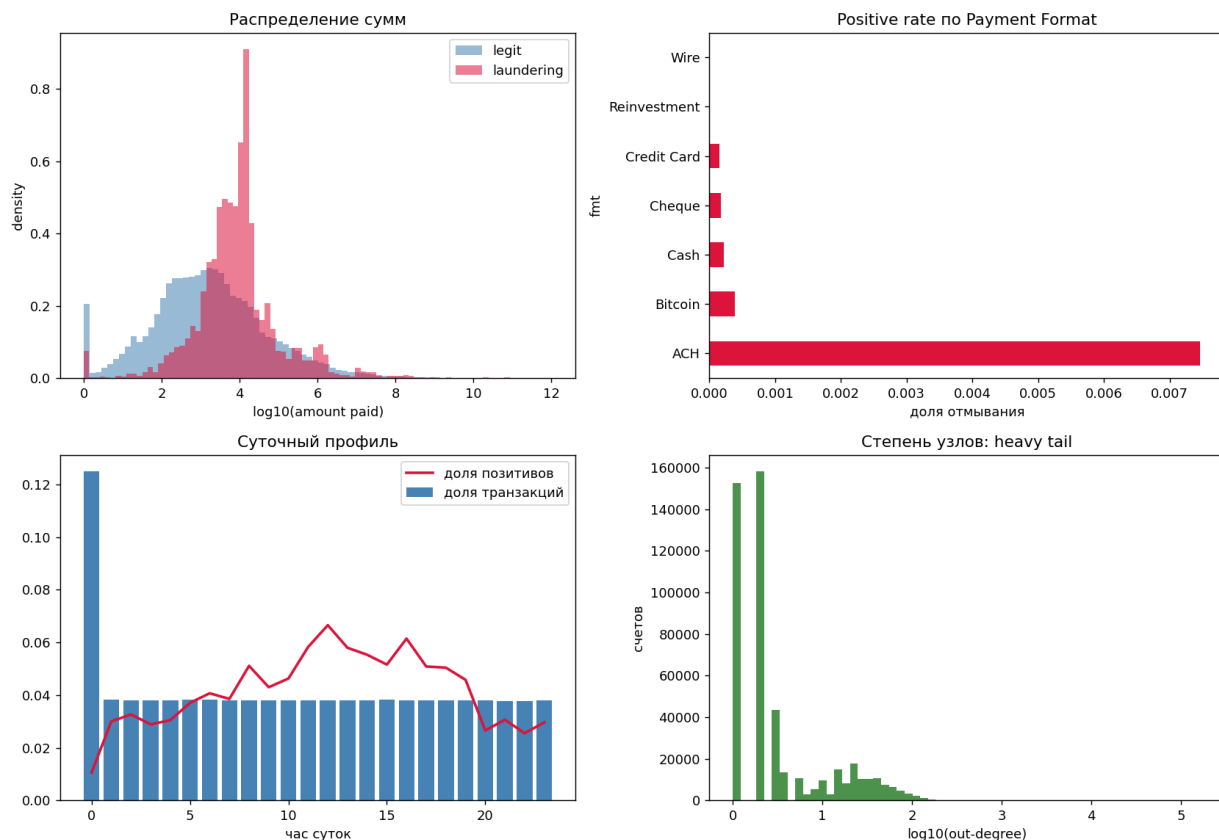


Рис. 1: Разведочный анализ. Распределение сумм в логарифмической шкале по классам; доля отмывания по каналам платежа; суточный профиль активности и позитивов; распределение исходящих степеней счетов.

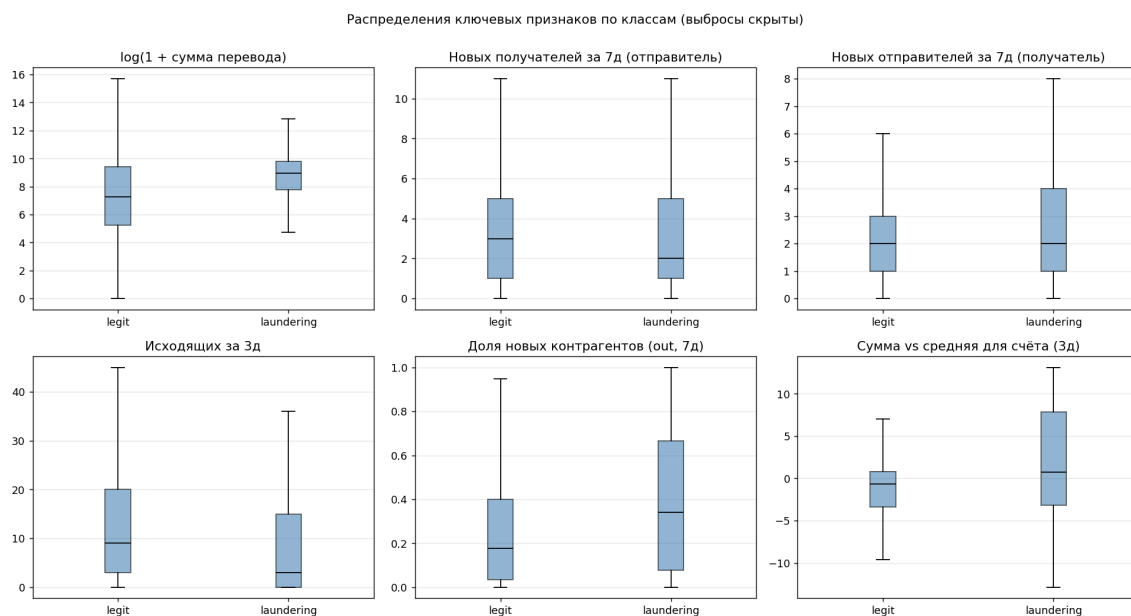


Рис. 2: Распределения ключевых построенных признаков по классам (выбросы скрыты). Медианы счётчиков новых контрагентов у позитивов заметно выше.

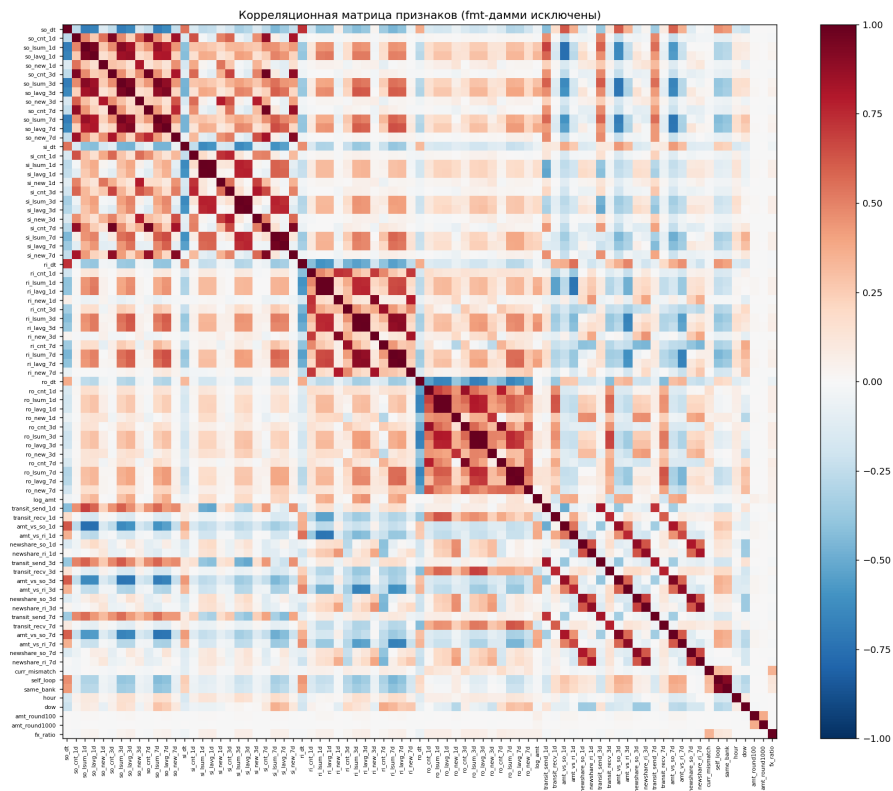


Рис. 3: Корреляционная матрица признаков. Три окна (1, 3, 7 дней) на каждое семейство ожидаемо скоррелированы; это обосновывает низкий `colsample_bytree = 0.42`, найденный подбором.

3 Feature engineering

Исходные 11 колонок почти не несут информации по отдельности: сигнал отмывания лежит в поведении счёта во времени и в топологии переводов. Построено 86 признаков.

3.1 Четыре профиля поведения

Каждая транзакция порождает два события: исходящее для отправителя и входящее для получателя. Для каждой транзакции запрашивается четыре профиля.

Префикс	Что описывает	Какую схему ловит
so_	отправитель: что рассылал раньше	fan-out, scatter
si_	отправитель: что получал раньше	приток перед рассылкой
ri_	получатель: что получал раньше	fan-in, gather
ro_	получатель: что рассылал раньше	транзитный счёт

Внутри каждого профиля считаются: количество транзакций в окне, логарифм суммы, логарифм среднего, число новых контрагентов и время с последнего события. Производные признаки: отношение объёмов входящих и исходящих (транзитность), отношение текущей суммы к типичной для счёта, доля новых контрагентов среди всех.

3.2 Вычислительная реализация

Наивный `groupby().rolling()` на 5 млн строк и 497 тыс. групп не помещается в память бесплатного Colab. Вместо него используется следующая схема:

1. события сортируются по паре (счёт, время) и кодируются одним `int64`-ключом $acct \cdot 2^{20} + ts$;
2. границы окна находятся двумя вызовами `np.searchsorted` сразу по всему массиву, без единого цикла по счетам;
3. суммы в окне берутся как разность префиксных сумм на границах.

Сложность $O(n \log n)$, пик памяти составляет сотни мегабайт, полное построение 86 признаков занимает 40 секунд.

3.3 Новые контрагенты вместо `nunique`

Точный `nunique` в скользящем окне через префиксные суммы не вычисляется, поскольку множество не аддитивно. Вместо него берётся число контрагентов, с которыми счёт раньше вообще не взаимодействовал: эта величина аддитивна, так как первое появление пары помечается единицей, и дальше работает обычный `cumsum`.

Замена оказалась не компромиссом, а одним из двух сильнейших семейств признаков: её удаление стоит -0.091 PR-AUC (раздел 9). Содержательно это объяснимо, поскольку взаимодействие со свежими счетами есть прямая подпись схем с подставными лицами.

3.4 Защита от утечки

Все окна строго полуоткрыты: `searchsorted(..., "left")` по метке времени исключает события той же минуты, включая саму текущую транзакцию. Ошибка на единицу здесь дала бы утечку таргета через собственную транзакцию.

Корректность проверена независимо: для 200 случайных транзакций значения признаков пересчитаны прямым перебором и сверены. Совпадение 200/200 по `so_cnt_1d` и `so_new_1d`.

4 Схема валидации

4.1 Почему не StratifiedKFold

Обычная стратифицированная кросс-валидация на этих данных даёт утечку из будущего: в один фолд попадают транзакции одной и той же отмывающей схемы, разнесённые во времени, и модель, обученная на её хвосте, предсказывает её же начало. Оценка получается завышенной и не переносится на продакшен, где модель всегда работает только с прошлым.

4.2 Expanding-window out-of-time

Окно обучения расширяется, валидация всегда на следующем дне. Формально это пять фолдов (требование задания), содержательно строгая OOT-схема.

Фолд	Обучение	Валидация
warmup	день 0	только история для окон, не обучаемся
1	дни 1–3	день 4
2	дни 1–4	день 5
3	дни 1–5	день 6
4	дни 1–6	день 7
5	дни 1–7	день 8
holdout	дни 1–8	день 9

Внутри каждого фолда последний день обучающего окна отдан под раннюю остановку, иначе выбор числа деревьев подглядывал бы в валидационный ответ. День 9 не участвует ни в одном фолде и не виден подбору гиперпараметров.

Все модели сравниваются по одной метрике на одном протоколе.

5 Классические модели

5.1 Логистическая регрессия (linear)

Бейзлайн. Негативы прорежены до 400 тыс. на фолд, позитивы взяты все, поскольку `StandardScaler` и `lbfgs` на полной матрице $3.7 \text{ млн} \times 86$ не помещаются в память. PR-AUC является ранговой метрикой и от прореживания негативов страдает слабо.

5.2 Random Forest (ensemble, бэггинг)

150 деревьев, глубина 14, `class_weight="balanced_subsample"`. Негативы прорежены до 300 тыс.: на двух vCPU бесплатного Colab полная выборка не обучается за разумное время. Это вычислительное ограничение, а не методическое решение, и оно учитывается при интерпретации.

5.3 XGBoost на GPU (ensemble, бустинг)

Здесь используется T4. `QuantileDMatrix` биннит признаки при создании матрицы, примерно 1 байт на значение вместо 4, что позволяет обучаться на полной выборке 3.7 млн строк без прореживания негативов.

Дисбаланс компенсируется через `scale_pos_weight`. SMOTE сознательно не используется: позитивы здесь не компактное облако в пространстве признаков, а восемь принципиально разных типов схем, и интерполяция между объектами разных схем породила бы синтетические точки в областях, где реальных объектов не бывает.

5.4 Подбор гиперпараметров

Optuna, TPE-сэмплер, 13 завершённых trial-ов при ограничении в 20 минут. Поиск ведётся на двух фолдах (одном «лёгком» и одном «тяжёлом»), чтобы оптимум не подгонялся под удачный день. Прирост относительно дефолтных параметров составил 33%.

Найденный профиль осмысленный: $\text{max_depth} = 10$ при $\text{min_child_weight} = 156$, $\text{colsample_bytree} = 0.42$, $\text{reg_lambda} = 28$. Глубокие деревья ради взаимодействий признаков, но с жёсткой регуляризацией, чтобы на 4.5 тыс. позитивов не уйти в запоминание.

Отдельно оптимизировался множитель к каноническому $\text{scale_pos_weight} = \text{neg}/\text{pos}$. Найденное значение **0.097** означает эффективный вес около 100 вместо канонических 1000. Полная компенсация дисбаланса раздувает recall в ущерб качеству ранжирования, на котором и строится PR-AUC.

6 Нейронная сеть (MLP)

6.1 Архитектура и обоснование

- **Embedding-слои** для категориальных признаков (формат платежа, час, день недели) вместо one-hot: сеть учит плотное представление, и близкие по поведению категории оказываются рядом в пространстве эмбедингов.
- **Три скрытых слоя 256, 128, 64.** Сужающаяся воронка: первый слой смешивает более 80 признаков, последующие сжимают до компактного представления. Больше слоёв не оправдано, поскольку на 4.5 тыс. позитивов глубокая сеть переобучится.
- **ReLU** как стандарт для табличных данных, не страдающий от затухания градиента.
- **Регуляризация тремя методами сразу** (задание требует минимум одного): **BatchNorm** после каждого линейного слоя, что стабилизирует обучение при разномасштабных признаках; **Dropout 0.3**; **weight decay 10^{-4}** в AdamW.
- **OneCycleLR** для разогрева и затухания learning rate за фиксированное число эпох.
- **VCEWithLogitsLoss** с тем же pos_weight , что у бустинга, ради сопоставимости.

Негативы прорежены до 600 тыс. на фолд по тем же причинам, что у Random Forest.

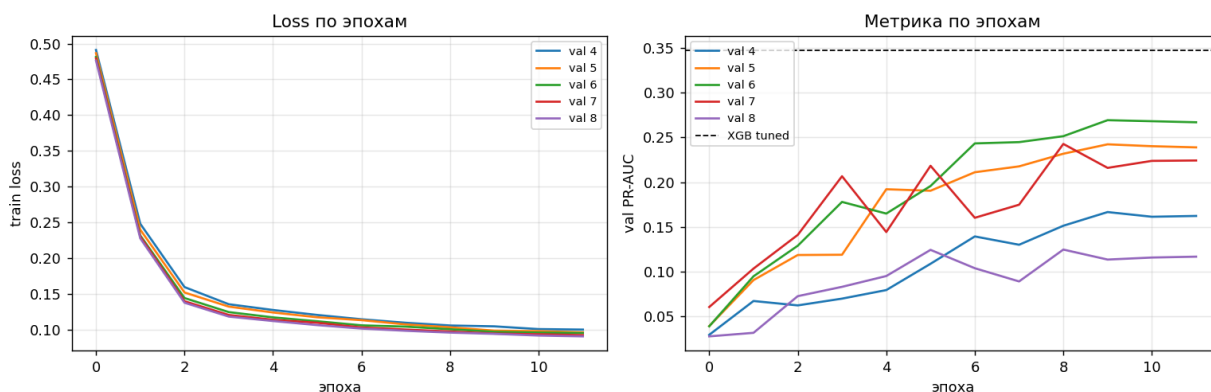


Рис. 4: Кривые обучения MLP по эпохам для всех пяти фолдов: train loss слева, валидационный PR-AUC справа. Пунктир показывает уровень настроенного XGBoost.

6.2 Что показывают графики обучения

Train loss выходит на плато примерно к шестой эпохе, а валидационный PR-AUC продолжает расти до последней. Сеть недообучена, а не переобучена, и отсутствие переобучения при 4.5 тыс. позитивов есть заслуга тройной регуляризации. При большем бюджете времени 25–30 эпох дали бы прибавку.

7 Сравнение моделей

Модель	PR-AUC	Без fmt	Lift	Обуч. строк	Время, с/фолд
naive LogReg (3 признака)	0.0112	0.0017	$\times 12.6$	400 тыс.	12
LogReg + FE	0.0359	0.0225	$\times 40.3$	400 тыс.	95
RandomForest + FE	0.1262	0.0647	$\times 141.7$	300 тыс.	195
MLP + FE	0.2019	0.1249	$\times 226.7$	600 тыс.	102
XGBoost + FE	0.2442	0.1381	$\times 274.2$	3.7 млн	14
XGBoost tuned + FE	0.3478	0.2282	$\times 390.5$	3.7 млн	61
Stacking (meta-LogReg)	0.3266	—	$\times 366.6$	—	0
Holdout, день 9	0.4617	—	$\times 217.6$	3.7 млн	—

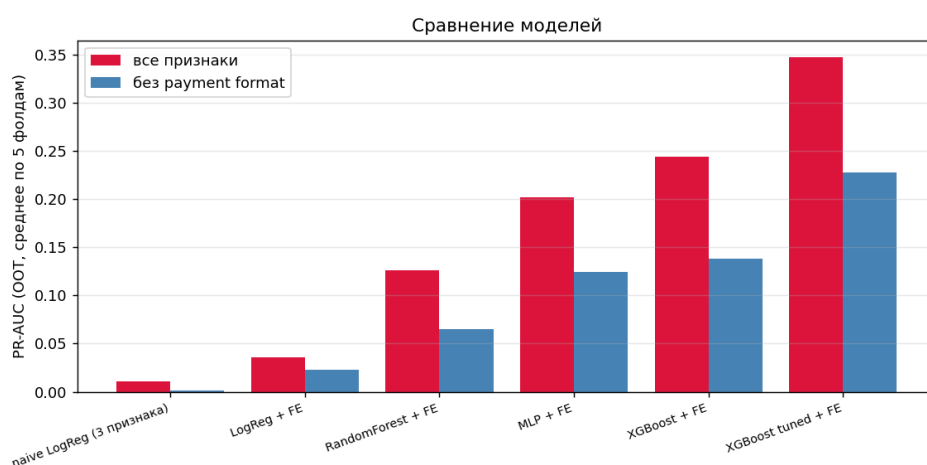


Рис. 5: Сравнение моделей в двух режимах. Разрыв между парой столбцов показывает вклад канала платежа: у наивного бейзлайна он составляет 6.5 раза, у финальной модели только 1.5.

7.1 Почему одна модель лучше другой

Градиентный бустинг против бэггинга. При дисбалансе 1:1122 бустинг на каждой итерации переносит вес на редкие ошибочные позитивы, тогда как бэггинг усредняет независимые деревья, обученные на бутстрап-выборках, в которых позитивов почти нет. Дополнительно XGBoost на GPU обучался на полной выборке, а Random Forest на прореженной в 12 раз, и при этом был втрое быстрее по секундомеру (61 с против 195 с на фолд), то есть примерно в 40 раз быстрее в пересчёте на строку.

Почему MLP проиграл бустингу. Признаки здесь счётчиковые, с длинными хвостами и естественными пороговыми эффектами. Условие вида «больше 15 новых контрагентов за 3 дня» дерево выражает одним разбиением, а сеть вынуждена аппроксимировать ступеньку гладкой функцией. При 4.5 тыс. позитивов на 86 признаков данных для такой аппроксимации мало. Это типичная картина для табличных задач и согласуется с публичными бенчмарками.

Почему линейная модель остановилась на 0.036. Корреляция каждого признака с таргетом по отдельности измеряется сотыми долями, что при таком дисбалансе ожидаемо. Весь сигнал лежит во взаимодействиях, которые линейная граница выразить не может. Отсюда же нестабильность логистической регрессии по фолдам: её качество плывёт в зависимости от состава конкретного дня.

Почему стекинг не помог. Оптимальный вес рангового бленда оказался равен 1.0, то есть любая примесь MLP строго вредит: 0.3478, 0.3372, 0.3304, 0.3242, 0.3107 при весах

XGBoost от 1.0 до 0.5. Мета-модель (0.3266) тоже не обошла одиночный бустинг. Причина в том, что MLP обучен на тех же признаках, слабее в 1.7 раза и ошибается там же, где бустинг. Ансамблирование помогает, когда модели ошибаются по-разному, а здесь это условие не выполняется.

Holdout выше среднего по фолдам (0.4617 против 0.3478), что означает отсутствие подгонки под валидацию. Разница объясняется длиной обучающей истории: на дне 9 модель видит восемь предшествующих дней против трёх-семи в ранних фолдах.

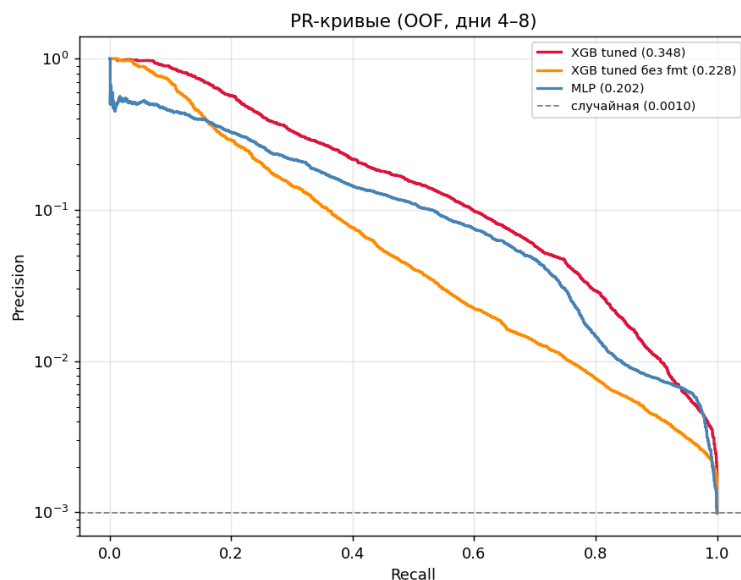


Рис. 6: PR-кривые на объединённых OOF-предсказаниях. Precision в логарифмической шкале.

8 Интерпретируемость: SHAP

Для задач AML интерпретируемость обязательна: по каждому алерту комплаенс-офицер должен понимать, почему перевод помечен подозрительным.

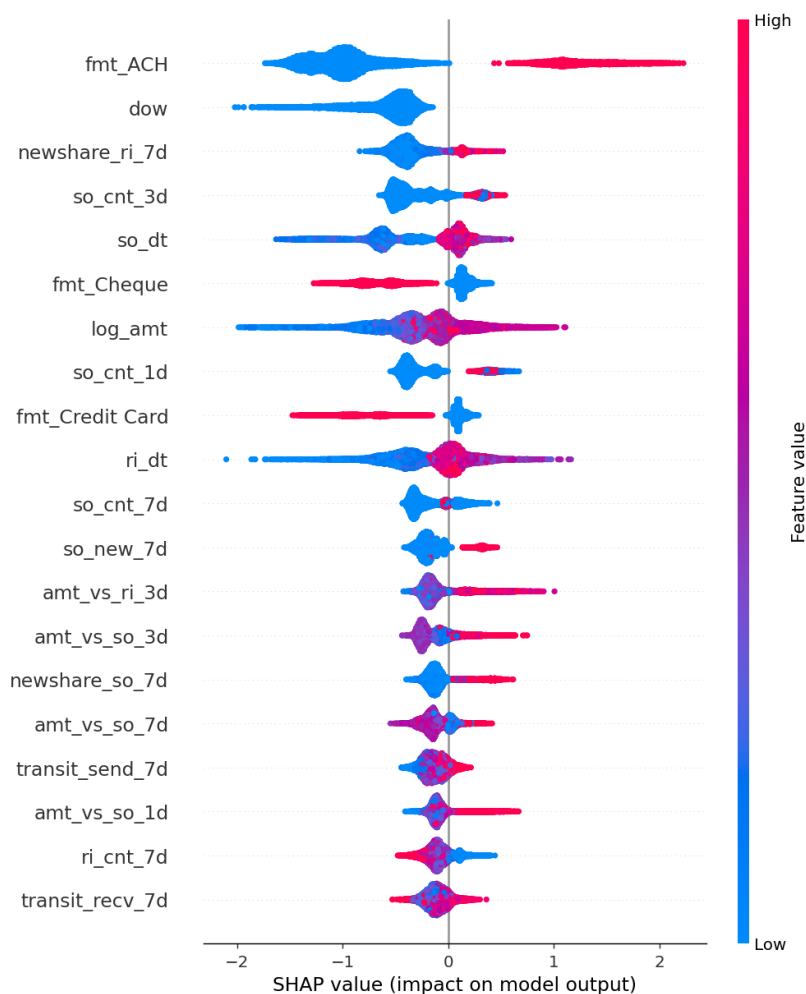


Рис. 7: SHAP summary plot на подвыборке дня 8. Высокие значения признаков (красное) сдвигают предсказание в сторону отмывания.

После канала платежа модель опирается на `newshare_ri_7d`, счётчики исходящих, `so_new_7d` и отношения текущей суммы к типичной. Все они работают в ожидаемую сторону.

Разбор конкретного алерта (транзакция с оценкой 0.993, метка 1) читается как готовое объяснение для аналитика: канал АСН, давно не было входящих переводов (`ri_dt`), всплеск новых отправителей за 3 и 7 дней (`ri_new_3d`, `ri_new_7d`), нехарактерно крупная сумма. Это классический fan-in на счёт подставного лица.

9 Дополнительное исследование

9.1 Error analysis по типам отмывающих схем

Датасет содержит файл разметки, где указано, какая транзакция к какой схеме относится. Эти метки никогда не попадают в признаки, поскольку это была бы прямая утечка таргета, и используются исключительно для анализа ошибок обученной модели. Сопоставление по составному ключу дало 100% совпадений на пригодных строках.

Схема	Позитивов	Recall	Топология
SCATTER-GATHER	325	0.618	звёздчатая
FAN-IN	158	0.589	звёздчатая
GATHER-SCATTER	287	0.568	звёздчатая
FAN-OUT	130	0.531	звёздчатая
RANDOM	96	0.354	смешанная
STACK	298	0.322	цепочечная
BIPARTITE	116	0.284	цепочечная
CYCLE	142	0.246	цепочечная
UNMATCHED	1000	0.075	периферия схем

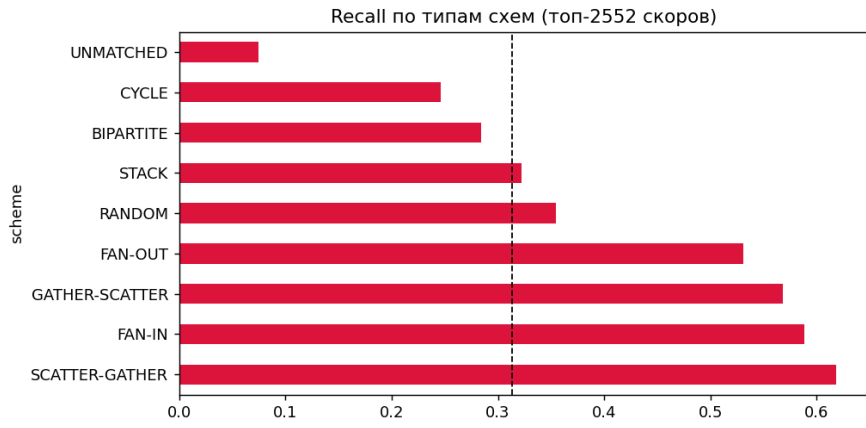


Рис. 8: Recall по типам схем при пороге, равном числу позитивов в пуле. Пунктир показывает общий recall 0.313.

Это главный содержательный результат работы. Схемы делятся на две чёткие группы, и граница проходит точно по топологии.

Четыре хорошо детектируемые схемы устроены одинаково: в них есть один узел, поведение которого аномально само по себе, когда счёт внезапно рассылает деньги многим или собирает от многих. Ровно это измеряют признаки `so_new_*` и `ri_new_*`.

Три плохо детектируемые схемы представляют собой многохоповые цепочки, где каждая отдельная транзакция выглядит совершенно рядовой, а подозрительна лишь последовательность переводов целиком. Ни один из 86 признаков не выходит за один хоп, поэтому провал структурный, а не случайный. Группа UNMATCHED (транзакции прикрытия вокруг ядер схем) с recall 0.075 подтверждает ту же логику.

Отсюда следует направление развития, и оно состоит не в увеличении модели: нужны 2-хор признаки или графовые нейронные сети. Увеличение глубины деревьев на этот класс схем не подействует, потому что информации о пути нет во входных данных.

9.2 Ablation по группам признаков

Важности показывают, что модель использует, но не отвечают на вопрос, что она потеряет без группы: скоррелированные признаки делят кредит между собой. Прямая проверка состоит в переобучении без каждого семейства (два фолда, базовое значение 0.3552).

Убрана группа	Δ PR-AUC	Признаков	Ранг по gain
payment format	-0.128	7	1
профиль получателя (ri_, ro_)	-0.098	26	5 и 6
новые контрагенты	-0.091	18	3
профиль отправителя (so_, si_)	-0.049	26	4 и 7
транзитность	-0.003	6	8

Два ранжирования расходятся, причём сильно. По gain признаки отправителя (3563) почти втрое обходят признаки получателя (1287), по ablation картина обратная: удаление стороны получателя стоит вдвое дороже. Причина в том, что gain распределяется между коррелирующими признаками пропорционально частоте выбора для разбиения, а не количеству информации в них. Сторона отправителя чаще оказывается первой под рукой и набирает gain, но её сигнал дублируется другими группами, тогда как сторона получателя набирает меньше и при этом незаменима.

Вывод согласуется с SHAP (верхние позиции у ri_dt, ri_new_3d, ri_new_7d) и с recall по схемам (FAN-IN 0.589 и GATHER-SCATTER 0.568 выше, чем FAN-OUT 0.531). На практике это значит, что feature_importance годится для беглого взгляда, но принимать по нему решение об удалении группы признаков нельзя.

Транзитность при этом добыта количественно: -0.0030 при разбросе по фолдам ± 0.074 , то есть неотличимо от нуля. Гипотеза «на счёте-прокладке втекает примерно столько же, сколько вытекает» не подтвердилась.

10 Бизнес-метрики

PR-AUC служит для сравнения моделей, но ничего не говорит бизнесу. Пропускная способность комплаенс-отдела фиксирована, поэтому содержательнее метрика Precision@K.

Алертов в день	Precision@K	Recall@K	Найдено схем в день
50	0.960	0.094	48.0
100	0.838	0.164	83.8
250	0.561	0.274	140.2
500	0.374	0.365	187.0
1000	0.230	0.448	229.6

При 100 алертах в день 84% из них оказываются настоящим отмыванием.

10.1 Почему наивная оптимизация порога вырождается

Стандартный подход состоит в минимизации ожидаемых издержек: стоимость разбора ложного срабатывания против потери от пропуска. На этих данных он не работает: формальный оптимум требует более 20 тысяч алертов в день, чего не разбирает ни один отдел.

Причина лежит в распределении сумм. Средняя сумма позитива составляет 36 млн при медиане 8.7 тыс., то есть несколько экстремальных транзакций формируют почти всю стоимость пропусков. При цене разбора 50 EUR модель приходит к выводу «проверяйте всё подряд». Это свойство оптимизации ожидаемых издержек на распределении с тяжёлым хвостом, и его стоит знать до того, как показывать такой график бизнесу.

10.2 Корректная постановка

Реальный банк не выбирает порог свободно, у него фиксирован штат. Правильная постановка звучит как «при N аналитиках какой объём отмывания мы перехватываем». Дополнительно суммы клипуются по 99-му перцентилю, чтобы единичные выбросы не определяли результат.

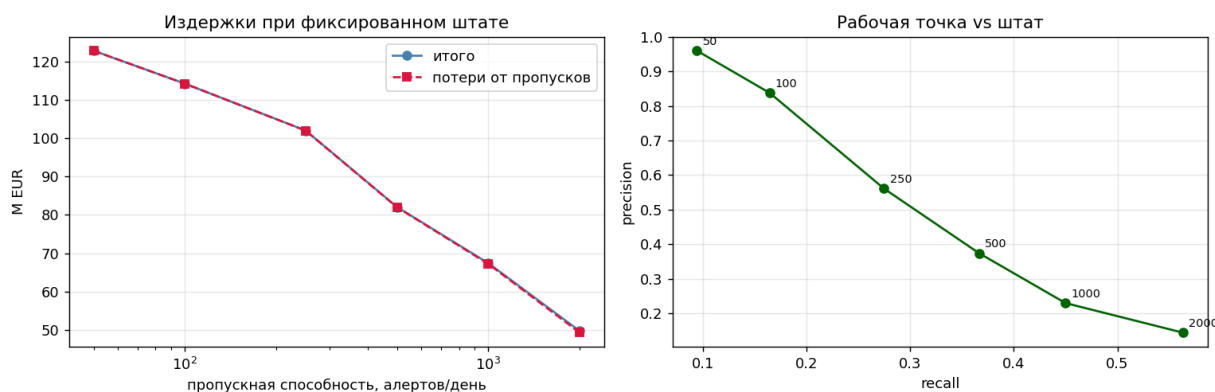


Рис. 9: Слева издержки при фиксированной пропускной способности, справа рабочая точка в координатах recall и precision с подписанным размером штата.

Кривые «итого» и «потери от пропусков» практически сливаются: затраты на разбор (0.5–428 тыс. EUR) на два-три порядка меньше потерь от пропусков (49–123 млн EUR). Узкое место здесь не стоимость ложных срабатываний, а пропускная способность. Модель в этой постановке не экономит на проверках, она ранжирует очередь, и каждый дополнительный аналитик окупается многократно.

Рекомендуемая рабочая точка составляет 250 алертов в день: precision 0.55, то есть каждый второй кейс настоящий, команда не выгорает на ложных срабатываниях и продолжает доверять скорам. При 500 и выше precision падает ниже 0.4, а это порог доверия к системе.

Отдельная оговорка: «экономия 63%» при 2000 алертах считается относительно сценария «не проверяем вообще ничего», а не относительно существующего rule-based процесса банка, и представляет собой нижнюю границу эффекта.

11 Выводы

1. **Прирост распадается на три множителя.** Ручные признаки дали $\times 3.2$ ($0.0112 \rightarrow 0.0359$), смена класса модели $\times 6.8$ ($\rightarrow 0.2442$), подбор гиперпараметров $\times 1.4$ ($\rightarrow 0.3478$). Второй множитель больше первого, но feature engineering от этого не становится второстепенным: колонка «без format» показывает $\times 134$ на одних только вручную построенных признаках, и без них бустингу было бы нечего комбинировать.
2. **Поведение счёта информативнее канала платежа.** Логистическая регрессия на признаках без формата платежа (0.0225) вдвое обгоняет наивный бейзлайн с ним (0.0112). Канал в этих данных почти детерминирован, что является артефактом AMLSim, поэтому честная оценка переносимости составляет 0.2282, а не 0.3478.
3. **Полная компенсация дисбаланса вредна.** Оптимальный `scale_pos_weight` около 100 при каноническом значении около 1000: агрессивное взвешивание раздувает recall в ущерб ранжированию, на котором и строится PR-AUC.
4. **Градиентный бустинг доминирует.** При дисбалансе 1:1122 он переносит вес на редкие ошибочные позитивы, тогда как бэггинг усредняет независимые деревья. MLP проигрывает из-за природы признаков: пороговые эффекты дерево выражает одним разбиением, а сеть аппроксимирует гладкой функцией.
5. **Модель видит узлы, но не видит пути.** Звёздчатые схемы детектируются с recall 0.53–0.62, цепочечные с 0.25–0.32, и граница по качеству совпадает с границей по топологии, поскольку все признаки ограничены одним хопом.
6. **Важности и ablation дают разные ответы.** Gain ставит признаки отправителя почти втрое выше признаков получателя, а удаление групп даёт обратную картину. Принимать решение об удалении группы признаков по `feature_importance` нельзя.
7. **Прикладная ценность измерима.** При 100 алертах в день 84% из них настоящее отмывание. Узкое место процесса это пропускная способность аналитиков, а не стоимость ложных срабатываний.

12 Ограничения

- **Синтетические данные.** Часть сигнала представляет собой артефакты генератора: у Wire и Reinvestment ровно ноль позитивов на 653 тыс. транзакций. На реальных данных ожидаемое качество ниже.
- **Нестабильность ООТ.** Разброс по фолдам ± 0.074 при среднем 0.348, то есть 21% относительной вариации. Проверка профиля валидационных дней смены режима не выявила, причина не установлена. В продакшене потребовалось бы ансамблирование по временным окнам.
- **Потолок в один хоп.** Признаки не выходят за непосредственных контрагентов счёта.
- **Вычислительные компромиссы.** Random Forest, MLP и логистическая регрессия обучались на прореженных негативах, полная выборка доступна только бустингу на GPU. Подбор гиперпараметров ограничен 13 trial-ами на двух фолдах.
- **Один датасет, один seed.** Переносимость не проверялась, доверительных интервалов нет.

13 Направления развития

1. **2-hop признаки:** реципрокные рёбра, замкнутость коротких путей, треугольники через булево умножение разреженных матриц на подграфе счетов с высокой степенью.
2. **Графовые нейронные сети** (GraphSAGE, GAT) для обучения представлений топологии вместо ручного конструирования.
3. **Аккаунт-центричная постановка:** классифицировать счета, а не переводы, поскольку отмывание является свойством счёта, а разметка на уровне транзакций размывает сигнал.
4. **Дообучение MLP:** кривые не вышли на плато, 25–30 эпох дали бы прибавку.
5. **Калибровка вероятностей,** если процесс потребует абсолютных порогов, а не ранжирования.